

INGENIERÍA Y CALIDAD DE SOFTWARE · 2026

Preparando para Producción

Actividad 4 — Docker productivo y Observabilidad

Proyecto AlentApp

Integrantes: Julián Coloma · Lucas Modernell · Shiroma Hajime · Tomás Rosato · Tomás Soler · Mariano Salas

1

CONTEXTO

El desafío: de desarrollo a producción

La aplicación corría en modo desarrollo: imágenes enormes, ejecutándose como root y con servidores de hot-reload. El objetivo fue dejarla lista para producción en dos frentes:

1

Docker productivo

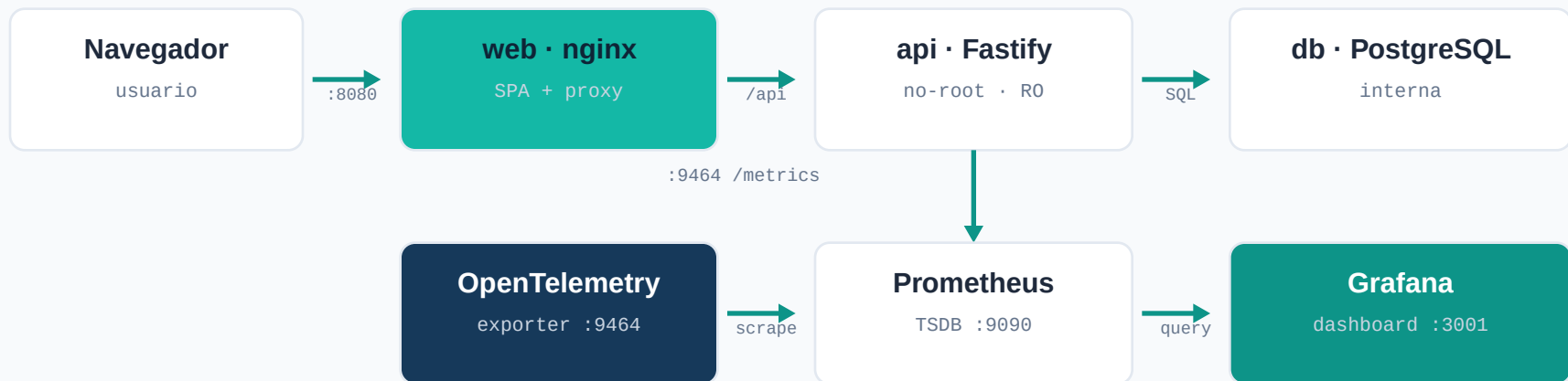
- Multi-stage builds -> imágenes chicas
- Usuario no-root, filesystem read-only
- nginx sirve el frontend; secretos en .env

2

Observabilidad

- OpenTelemetry instrumenta la API
- Métricas RED (Rate, Errors, Duration)
- Prometheus + Grafana para visualizar

Arquitectura final del sistema

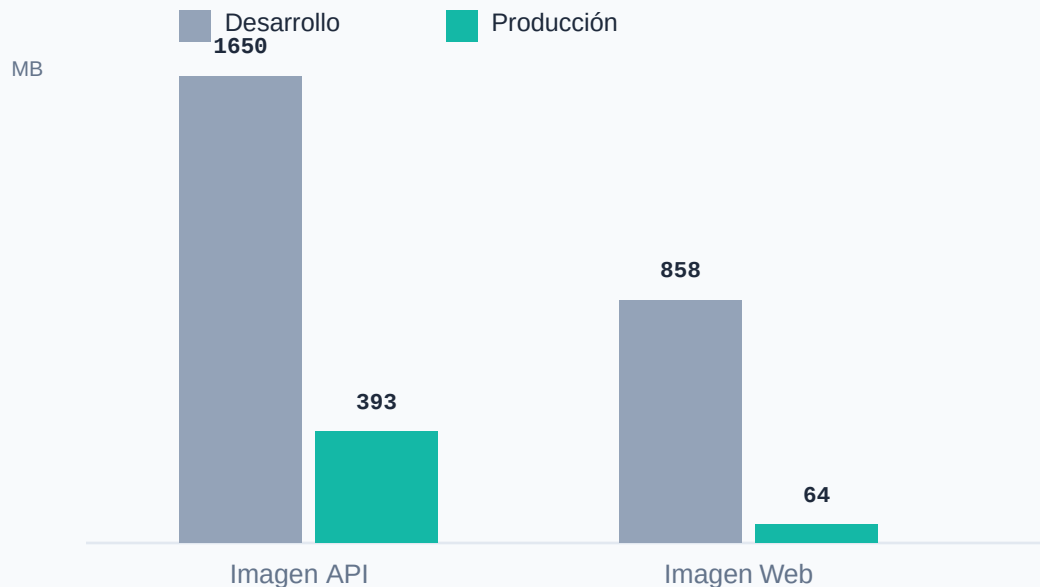


Única entrada pública: nginx en :8080. La API, la base de datos y la observabilidad viven en una red interna aislada (alentapp_prod_net).

3

RESULTADOS

Antes y después: tamaño y startup

**-76%****Imagen API**

1.65 GB -> 393 MB

-92%**Imagen Web**

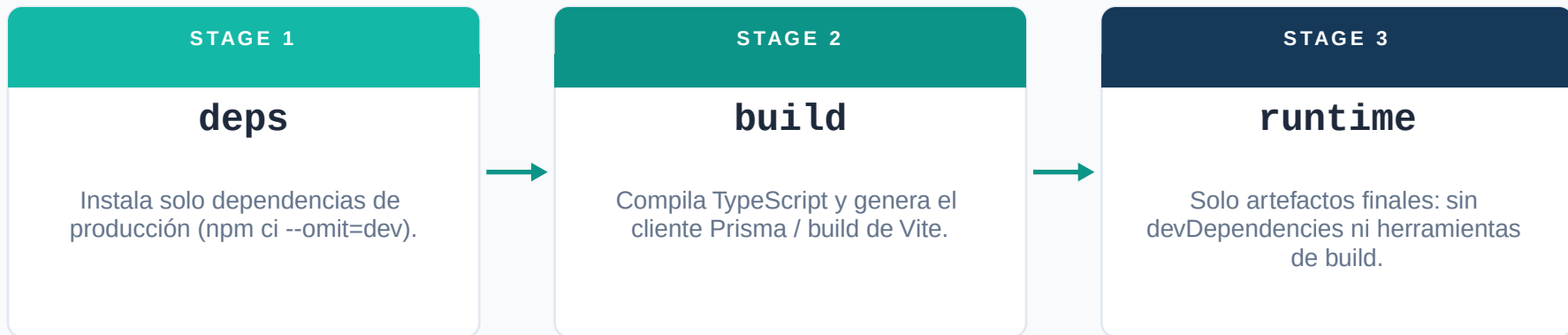
858 MB -> 64 MB

22 s**Startup del stack**

de cero hasta healthy

Objetivo del TP: reducir al menos 70 %. Cumplido con multi-stage builds.

Multi-stage builds + nginx



El frontend no usa Node en producción: nginx sirve los archivos estáticos del build de Vite (con gzip, cache de assets y security headers) y hace de reverse-proxy de /api hacia la API.

Defensa en profundidad

Usuario no-root

El proceso corre como 'node', no como root: menor impacto si lo comprometen.

Filesystem read-only

read_only + tmpfs: el contenedor no se puede modificar en runtime.

Capabilities mínimas

cap_drop: ALL + no-new-privileges: se quitan permisos del kernel.

Secretos en .env

Credenciales fuera del código y de git; 0 hardcodeadas en el compose.

DB no expuesta

PostgreSQL solo accesible por la red interna, sin puerto al host.

Healthchecks

Docker reinicia el servicio si deja de responder; arranque ordenado.



OBSERVABILIDAD

Dashboard RED — 6 paneles

Pipeline: API (OpenTelemetry) -> :9464 -> Prometheus -> Grafana

1 · Requests por segundo

Tráfico (Rate)

2 · Tasa de error %

Errores 4xx/5xx (Errors)

3 · Latencia p95 / p99

Performance (Duration)

4 · Por status code

Distribución de respuestas

5 · Memoria del proceso

Consumo de recursos

6 · Endpoints más lentos

Cuellos de botella

Verificado: los 6 paneles responden al tráfico real y la tasa de error refleja los 4xx/5xx generados.



DEMO EN VIVO

Dashboard RED en acción

- Levantar el stack: `docker compose -f docker-compose.prod.yml up -d`
- Generar tráfico contra la API (incluyendo errores 4xx)
- Abrir Grafana en `http://localhost:3001`
- Mostrar los 6 paneles respondiendo en tiempo real
- Ver el pico en "Tasa de error %" tras los 4xx

[Insertar captura del dashboard
"RED — AlentApp API" con datos reales]

Lo más difícil y qué cambiaríamos

Lo más difícil

- package-lock.json desincronizado con las deps de OpenTelemetry: el build fallaba.
- Errores que solo aparecen en prod: el build usa tsc (estricto), pero dev usa tsx (sin type-check).
- read-only rompía el arranque (la API escribe en / uploads): resuelto con un volumen.
- Coordinar el trabajo en paralelo entre integrantes y ramas.

Qué cambiaríamos

- Imagen base distroless para la API (quitar npm del runtime y achicar más).
- Migraciones de Prisma como servicio automático en el compose, no manual.
- OTLP + OpenTelemetry Collector en vez del exporter Prometheus directo.
- CI que corra los tests y el build antes de cada merge.

RESULTADOS

Un sistema productivo, seguro y observable

- ✓ Imágenes -76 % (API) y -92 % (Web) con multi-stage builds
- ✓ Seguridad por capas: no-root, read-only, capabilities mínimas, secretos en .env
- ✓ Observabilidad RED end-to-end: OpenTelemetry !' Prometheus !' Grafana
- ✓ Stack que levanta de cero, queda healthy en 22 s y es funcional

¡Gracias! ¿Preguntas?